

CPU Reordering, Instruction-Level Parallelism & Memory Ordering

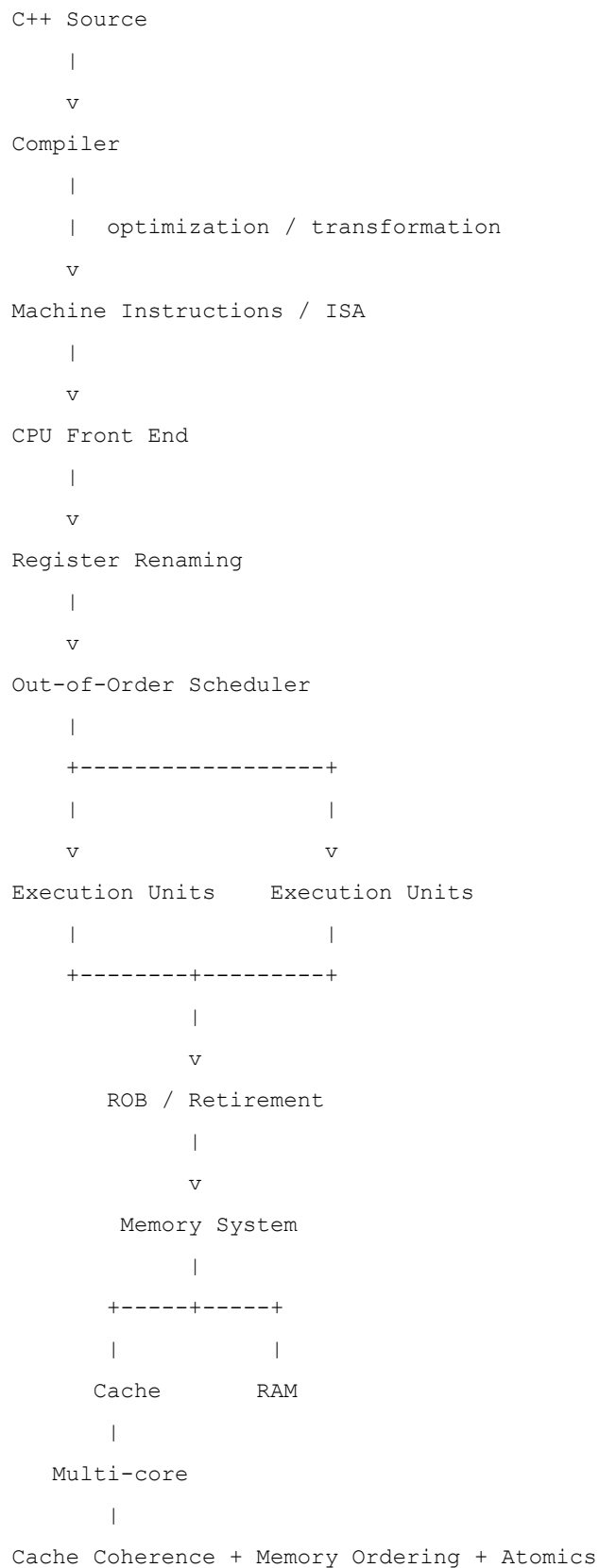
1. Objective

This study investigates how modern CPUs execute instructions and how compiler optimization, out-of-order execution, cache/memory behavior, and concurrency memory ordering differ.

The central distinction is:

Compiler reordering is a compile-time transformation, out-of-order (OoO) execution is a runtime CPU mechanism, and memory ordering defines the guarantees that concurrent threads can rely upon when observing memory operations.

2. Core Mental Model



Important layers:

1. **Compiler transformation** — changes the generated instruction sequence while preserving the required language-level behavior.

2. **ISA** — defines the machine-level instruction and memory-ordering contract.
 3. **CPU microarchitecture** — determines how instructions are actually fetched, scheduled, executed, speculated, and retired.
 4. **Memory ordering** — determines which memory-operation orderings concurrent software can rely upon.
 5. **Cache coherence** — handles coherent views of cache lines across cores.
-

3. Experiment 1 — Compiler Optimization

Source

```
#include <iostream>

int foo(int a, int b) {
    int x = a + 1;
    int y = b + 2;
    return x * y;
}

int main() {
    std::cout << foo(10, 20) << '\n';
}
```

Commands used:

```
g++ -O0 -S -masm=intel experiment1.cpp -o experiment1_O0.s
g++ -O2 -S -masm=intel experiment1.cpp -o experiment1_O2.s
```

—O0

Relevant assembly:

```

foo:
    mov DWORD PTR 16[rbp], ecx
    mov DWORD PTR 24[rbp], edx

    mov eax, DWORD PTR 16[rbp]
    add eax, 1
    mov DWORD PTR -4[rbp], eax

    mov eax, DWORD PTR 24[rbp]
    add eax, 2
    mov DWORD PTR -8[rbp], eax

    mov eax, DWORD PTR -4[rbp]
    imul eax, DWORD PTR -8[rbp]

```

Conceptually:

```

a
|
v
stack -> register -> +1 -> stack
                        |
                        v
                        x

b
|
v
stack -> register -> +2 -> stack
                        |
                        v
                        y

x * y -> result

```

At -O0, GCC keeps intermediate variables in memory and creates a stack frame.

-O2

Relevant assembly:

```
foo:
    lea eax, 1[rcx]
    add edx, 2
    imul eax, edx
    ret
```

Interpretation:

```
lea eax, 1[rcx]      ; EAX = a + 1
add edx, 2           ; EDX = b + 2
imul eax, edx        ; EAX = (a + 1) * (b + 2)
ret
```

`lea` is being used as an arithmetic instruction here. It is not loading from memory.

Additional optimization observed

The optimized `main` contained:

```
mov edx, 242
```

instead of calling `foo(10, 20)`.

The compiler could calculate:

```
(10 + 1) * (20 + 2)
= 11 * 22
= 242
```

at compile time.

This is **constant folding**.

Conclusion

At `-O0`, GCC generated a relatively direct implementation containing stack operations and intermediate memory locations. At `-O2`, GCC eliminated unnecessary memory accesses, kept values in registers, simplified the computation, and even evaluated `foo(10, 20)` at compile time. This demonstrates compiler optimization and should not be confused with runtime CPU out-of-order execution.

4. Experiments 2 & 3 — Instruction-Level Parallelism and Dependencies

These experiments were performed together.

Experiment 2: Independent operations

Source:

```
int independent(int a, int b, int c, int d) {  
    int x = a + b;  
    int y = c + d;  
    return x * y;  
}
```

Relevant assembly:

```
lea eax, [rcx+rdx]  
add r8d, r9d  
imul eax, r8d  
ret
```

The two additions are independent:

```
a + b -----+  
              |  
              v  
            multiply  
              ^  
              |  
c + d -----+
```

The compiler produced:

```
lea eax, [rcx+rdx]    ; x = a + b
add r8d, r9d          ; y = c + d
imul eax, r8d         ; x * y
```

Because the additions do not depend on each other, a superscalar OoO CPU can potentially execute them using different execution resources or overlap their execution.

This is **instruction-level parallelism (ILP)**.

Experiment 3: Dependent operations

Source:

```
int dependent(int a, int b) {
    int x = a + b;
    int y = x + 10;
    int z = y * 2;
    return z;
}
```

Relevant optimized assembly:

```
lea eax, 10[rcx+rdx]
add eax, eax
ret
```

GCC simplified the original computation to:

```
(a + b + 10) * 2
```

The conceptual dependency remains:

```
a + b
  |
  v
+ 10
  |
  v
x 2
```

or:

```
I1 -> I2 -> I3
```

This is a true **RAW (Read After Write)** dependency.

The second operation needs the value produced by the first, and the third needs the value produced by the second.

Combined conclusion

The experiments demonstrated the difference between independent and dependent instructions. In the independent example, $a + b$ and $c + d$ did not depend on each other, allowing a superscalar out-of-order CPU to potentially execute them in parallel or overlap their execution. In the dependent example, each computation depended on the previous result, creating a RAW dependency that limits instruction-level parallelism.

The compiler also optimized the dependent computation by eliminating intermediate variables and reducing the number of instructions. Therefore:

Independent instructions

|

v

More ILP available

|

v

Superscalar / OoO CPU can exploit it

True RAW dependency

|

v

Required value must become available

|

v

Limits parallelism

A key lesson is:

OoO execution can exploit independent work, but it cannot eliminate a true data dependency.

5. Experiment 4 — Branches and Compiler Optimization

Source

```
int branch_test(int x) {  
    if (x > 100)  
        return x + 10;  
    else  
        return x - 10;  
}
```

Commands:

```
g++ -O0 -S -masm=intel experiment3.cpp -o experiment3_O0.s  
g++ -O2 -S -masm=intel experiment3.cpp -o experiment3_O2.s
```

—O0

Relevant assembly:

```
cmp DWORD PTR 16[rbp], 100  
jle .L2  
  
mov eax, DWORD PTR 16[rbp]  
add eax, 10  
jmp .L3  
  
.L2:  
mov eax, DWORD PTR 16[rbp]  
sub eax, 10  
  
.L3:  
pop rbp  
ret
```

This contains a real conditional branch:

```
jle .L2
```

Conceptually:

```
      cmp x, 100
      |
+-----+-----+
|               |
| x > 100        | x <= 100
|               |
| x + 10         | x - 10
```

A CPU executing this branch may use its branch predictor to predict which path will be needed.

-02

Relevant assembly:

```
lea edx, 10[rcx]
lea eax, -10[rcx]
cmp ecx, 101
cmovge eax, edx
ret
```

Instead of using a conditional jump, GCC computes both possible values and uses:

```
cmovge
```

which is a conditional move.

Conceptually:

```
x + 10 -----+
              |
              v
          cmovge ---> result
              ^
              |
x - 10 -----+
```

There is no conditional branch for this decision.

Conclusion

At `-O0`, GCC generated a conditional jump corresponding closely to the C++ `if/else`. At `-O2`, GCC transformed the control flow into a conditional move. This demonstrates that compiler optimization can change the control-flow structure before execution.

It also demonstrates the distinction between compiler optimization and branch prediction:

Compiler:

May eliminate/change a branch

CPU:

If a branch exists, may predict its outcome at runtime

6. Experiment 5 — C++ Atomics and Memory Ordering

Release/Acquire program

```

#include <atomic>
#include <iostream>
#include <thread>

int data = 0;
std::atomic<bool> ready{false};

void producer() {
    data = 42;
    ready.store(true, std::memory_order_release);
}

void consumer() {
    while (!ready.load(std::memory_order_acquire)) {
    }

    std::cout << data << '\n';
}

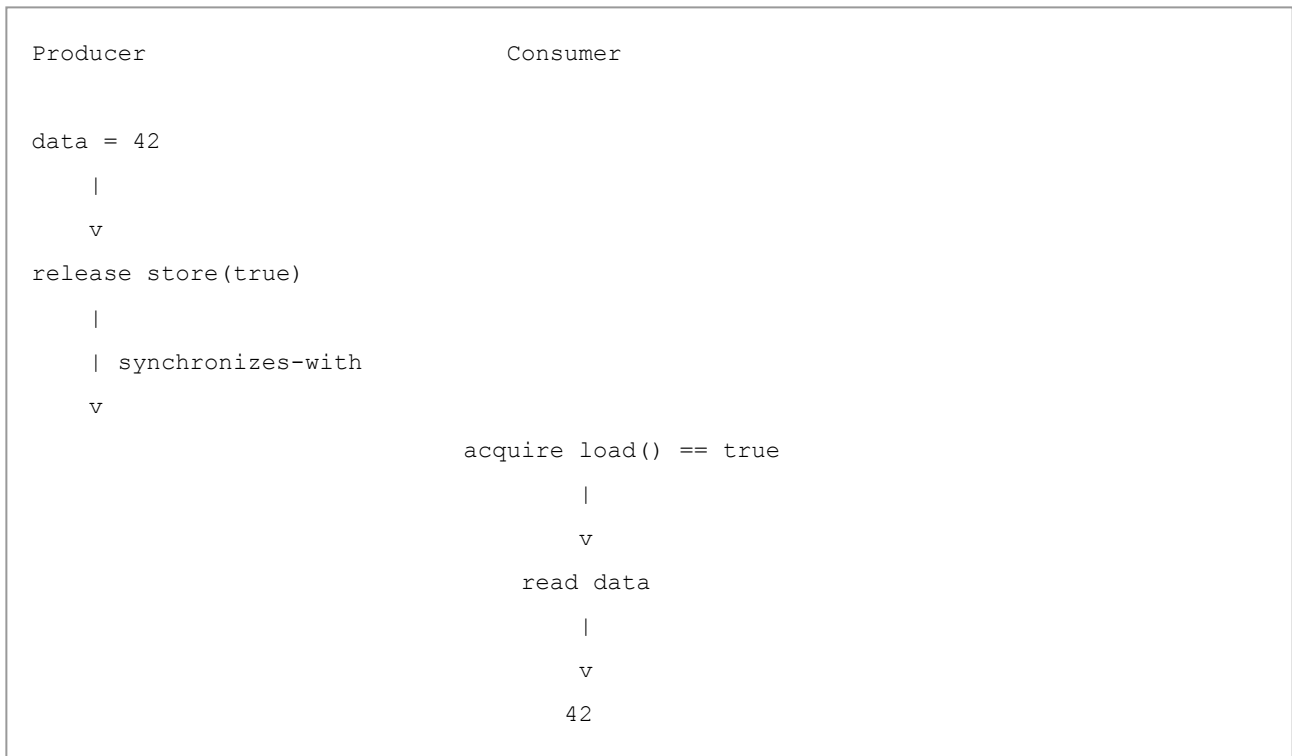
int main() {
    std::thread t1(producer);
    std::thread t2(consumer);

    t1.join();
    t2.join();
}

```

The program was compiled, but execution was initially blocked by Windows Application Control. The execution limitation is environmental and does not change the memory-model analysis.

The release/acquire relationship can be represented as:



If the acquire load observes the value written by the release store, the release/acquire synchronization establishes the required happens-before relationship for the preceding write to `data`.

7. Relaxed Version

The relaxed version changed the operations to:

```
ready.store(true, std::memory_order_relaxed);
```

and:

```
ready.load(std::memory_order_relaxed);
```

The program produced:

```
42
```

Important interpretation

The observed 42 does **not** prove that relaxed ordering provides the same guarantee as release/acquire.

A runtime observation means:

Observed:

"This particular execution printed 42."

It does NOT mean:

"The C++ memory model guarantees 42 in all executions."

Furthermore, this exact relaxed example is not a clean experiment for demonstrating a different output because `data` is non-atomic. Removing the release/acquire synchronization can leave the conflicting access to `data` without a proper happens-before relationship, resulting in a C++ data race and undefined behavior.

Therefore, the correct conclusion is about **guarantees**, not about trying to make the relaxed program print a different value.

8. Atomicity vs Ordering

These concepts must not be confused.

Atomicity

|

+-- Individual atomic operation has atomic semantics

Memory ordering

|

+-- Defines ordering/synchronization relationships

Cache coherence

|

+-- Coordinates coherent views of cache lines across cores

An atomic variable being atomic does not automatically synchronize unrelated non-atomic data.

For example:

```
data = 42;
ready.store(true, std::memory_order_release);
```

paired with:

```
if (ready.load(std::memory_order_acquire))  
    print(data);
```

establishes a synchronization relationship.

Relaxed operations do not provide that same release/acquire synchronization.

9. Memory Ordering Summary

Ordering

Main property

relaxed	Atomic operation with minimal ordering guarantees
release	Publishes preceding operations when paired appropriately with acquire
acquire	Establishes ordering after observing the corresponding release
acq_rel	Acquire + release semantics for suitable read-modify-write operations
seq_cst	Strong sequentially consistent ordering for seq-cst atomic operations

Release/acquire mental model

```
RELEASE  
"Publish what I did before this."  
  
    |  
    v  
  
ACQUIRE  
"If I observe that publication,  
I can safely use what was published."
```

10. Sequential Consistency

A useful classic example is:

Initial:

```
x = 0
y = 0
```

Thread 1:

```
x.store(1, std::memory_order_seq_cst);
r1 = y.load(std::memory_order_seq_cst);
```

Thread 2:

```
y.store(1, std::memory_order_seq_cst);
r2 = x.load(std::memory_order_seq_cst);
```

With sequential consistency, all seq-cst operations must fit into one globally consistent order while respecting each thread's program order.

Therefore:

```
r1 = 0
r2 = 0
```

is not permitted.

For example:

```
x.store(1)
  |
  v
r1 = y -> 0
  |
  v
y.store(1)
  |
  v
r2 = x -> 1
```

gives:

```
r1 = 0
r2 = 1
```

which is valid.

The important mental model is:

Sequential consistency provides a single globally consistent ordering of the sequentially consistent atomic operations. It does not mean the hardware literally executes all instructions one at a time.

11. Store Buffers and Hardware Intuition

A simplified CPU model is:

```
Core
|
v
Store Buffer
|
v
Cache
|
v
Memory
```

A store can be temporarily held in a store buffer while the core continues doing other work.

This helps explain why weakly ordered systems can produce observations that differ from a simplistic "every store immediately reaches memory" model.

However:

A store-buffer diagram is a hardware intuition, not a complete description of every CPU.

Actual behavior depends on the ISA and microarchitecture.

12. Compiler Reordering vs CPU Execution vs Memory Ordering

This is the central conclusion of the entire study.

Compiler transformation

The compiler may transform:

```
Source program
|
v
Equivalent machine-code sequence
```

Examples from the experiments:

- Eliminating stack variables
- Constant folding
- Replacing a branch with `cmov`
- Combining arithmetic operations

Out-of-order execution

The CPU can dynamically execute ready instructions when their operands and execution resources are available.

Example:

```
I1: slow load
I2: independent add
I3: depends on I1
I4: independent add
```

Potentially:

```
I2 -> I4 -> I1 -> I3
```

as execution timing, while retirement can preserve program order.

Memory ordering

Memory ordering determines what ordering relationships concurrent threads can rely upon.

It is not simply:

"Which instruction physically executes first?"

Instead, it is about the guarantees concerning memory operations and observations between threads.

Final distinction

Compiler reordering

= compile-time transformation

OoO execution

= runtime microarchitectural scheduling

Memory ordering

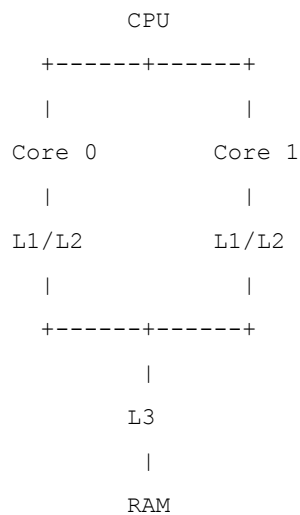
= rules governing permitted memory-operation observations

A concise formulation is:

Compiler reordering and OoO execution are mechanisms at different layers, while memory ordering is the rulebook defining which memory-operation orderings concurrent software is allowed to observe and rely upon.

13. Cache and Multi-Core Context

Modern cores have private cache levels and share larger levels:



Important concepts:

Cache hit

Requested data is found at the current cache level.

Cache miss

Data is not found and must be obtained from a lower level.

Cache line

Caches transfer data in blocks rather than treating every individual byte as an isolated cache object.

Spatial locality

Nearby memory locations are likely to be accessed together.

Example:

```
array[0]
array[1]
array[2]
array[3]
```

Sequential access benefits from cache lines.

Temporal locality

Recently used data is likely to be used again.

Cache coherence

Multiple cores may cache the same memory location. Coherence mechanisms maintain coherent cache-line state across cores.

However:

Cache coherence and memory ordering are different concepts.

Coherence does not by itself provide the synchronization guarantees required by the C++ memory model.

14. Overall Findings

The experiments support the following conclusions:

1. Compiler optimization happens before runtime.

GCC can eliminate memory operations, fold constants, combine arithmetic, and transform branches.

2. Instruction-level parallelism depends on dependencies.

Independent operations can potentially execute in parallel; true RAW dependencies limit parallelism.

3. OoO execution is not the same as compiler reordering.

The compiler transforms the instruction stream before execution. The CPU dynamically schedules instructions at runtime.

4. Branch prediction is a runtime CPU mechanism.

It helps keep the pipeline supplied with useful instructions when conditional branches exist.

5. Compiler optimization can eliminate branches.

The branch experiment demonstrated GCC replacing a conditional jump with `cmovge`.

6. Cache coherence is not memory ordering.

Coherence concerns consistent cache-line state; memory ordering concerns ordering and synchronization guarantees.

7. Atomicity is not the same as synchronization.

Relaxed atomics provide atomic operations without the release/acquire synchronization relationship.

8. Release/acquire can establish happens-before.

When an acquire operation observes the appropriate release operation, preceding writes can safely be ordered before subsequent operations.

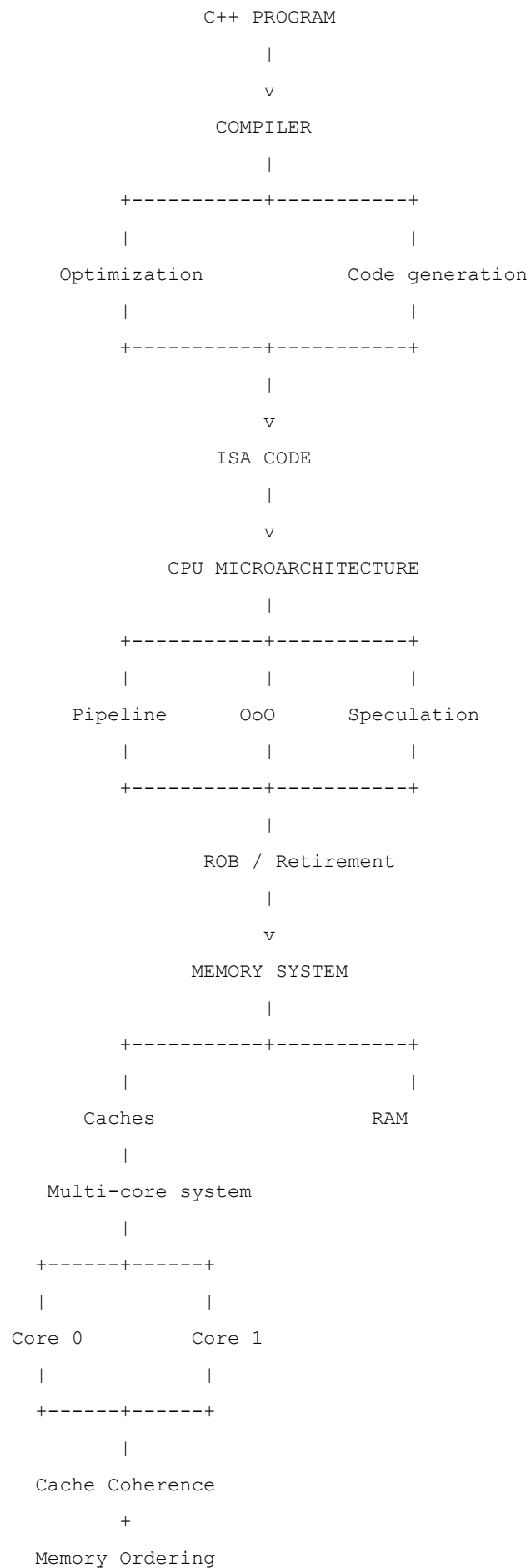
9. A runtime experiment cannot prove a language-level guarantee.

Observing 4.2 on a relaxed program does not prove that 4.2 is guaranteed by the C++ memory model.

10. The C++ memory model is the programmer-facing contract.

Hardware behavior, cache implementations, store buffers, and CPU scheduling are implementation mechanisms; portable C++ concurrency must be reasoned about using the language's defined memory-order guarantees.

15. Final Mental Model



Final takeaway

A modern CPU system must be understood at multiple layers. The compiler can transform source code, the CPU can execute independent instructions out of order, caches and coherence determine how data moves through the memory hierarchy, and the language/ISA memory model determines which ordering guarantees concurrent programs can rely upon. Correct concurrent programming therefore cannot be based solely on the apparent source-code order or on observations from one hardware run.

